SMS Project

Smart Machine Systems, summer 2026

This document is to introduce you into the technical details of the project task for this years SMS course.

Assumptions

1. You have access to a Windows/Mac/Linux computer, optimally with 16 GB+ memory (RAM)

Installation

- **If you are on Windows or MacOs, follow these steps.**

1. Install a virtual machine (VM) of Ubuntu 22.04 LTS (https://releases.ubuntu.com/jammy/)
   - If you do not have a virtual machine manager software installed, install Oracle VirtualBox (https://www.virtualbox.org/)
   - An exemplary guide is https://itslinuxfoss.com/install-ubuntu-22-04-virtualbox/, with many more on the internet
   - As the robot will be simulated inside of this VM, it needs some compute power: Around 8 GB of RAM and 30 GB of storage should work
2. Start the virtual machine and create an arbitrary user
   - User name is not relevant for us, please remember the password!
3. **Inside the VM**: Open a terminal
4. Install git and VSCodium[1]: `sudo apt install git codium`
   - Whenever you enter `sudo`, you are doing a privileged (admin) action. You will be prompted to enter the password of the account inside the VM. Typically, you can't see the characters you are entering, but they exist :)
5. Download our software package:

   ```
   git clone https://github.com/ma4096/unitree_sdk2_docker.git
   ```

6. Install the software (execute line by line):

   ```
   cd unitree_sdk2_docker
   chmod +x ./install_mono
   sudo ./install_mono
   ```

- **If you are on Linux natively**, follow the steps outlined in the README.md of the `unitree_sdk2_docker` repo (short installation). Usage is a bit different, but you can use `./start_[docker, mujoco]` to launch the SDK or MuJoCo containers respectively.
  - Using `./install_mono` works too, but can make changes to your local system. If you want to use it anyway, consider sourcing a Python virtual environment before running the installation.

[1] VSCodium is a "version" (fork) of Virtual Studio Code without any Microsoft associations. Both are open-source software. For the most part they are interchangable, but Codium is easier to install.

Orientation

The software consists of two main pieces:

- The software you write, supported by our Python libraries that are now installed.
- The simulation software (MuJoCo) that communicates with your software just like the real robot would do. For this simulation, you will build the simulation scene to resemble our lab.

The now installed folders should look like:

```
unitree_sdk2_docker/
├── code/
│   └── all your stuff should happen here :)
├── unitree_mujoco/
│   ├── simulate_python/
│   │   ├── config.py
│   │   └── unitree_mujoco.py
│   ├── unitree_robots/
│   │   ├── scene.xml
│   │   └── ...
│   └── ...
├── isem_go2_interface/
│   ├── src/
│   │   └── isem_go2_interface/
│   │       ├── Go2.py
│   │       ├── OnnxController.py
│   │       └── ...
│   └── ...
└── (other folders not directly relevant to you)
```

Building the simulation

We use a modified simulation environment based on Unitree's official MuJoCo version (https://github.com/unitreerobotics/unitree_mujoco) which integrates more sensors.

> 💡 **What is MuJoCo?**
>
> MuJoCo (Multi-Joint dynamics with Contact) is a simulation package for rigid robotics originally developed by Google. Like all robotic simulation tools, it is takes inputs (like torque applied to a joint), simulates the physical reaction of the system, and returns some defined sensor readings. For us, it is also important that the system can be visually observed.

To start the simulation, run

```
python unitree_mujoco.py
```

inside the `unitree_sdk2_docker/unitree_mujoco/simulate_python` folder. An empty world with only the robot and some boxes will show up. Look, zoom, and move around using the mouse and explore the settings to make yourself familiar with the interface.

This scene is defined in `unitree_sdk2_docker/unitree_mujoco/unitree_robots/go2/scene.xml`. Edit the boxes that are in the scene by changing some of their properties inside the `<worldbody>` tag. After saving the file and restarting the simulator, you can observe the changes.

Experiment with these properties, create new bodies, and look up how to include CAD models (we observed it to be easiest with `.obj` files).

Inside this `scene.xml` file, you will build the simulation environment in which the robot will move according to the code you write.

Programming

In the **code** directory, your code lives. When we deploy the software on the real robot, we will copy everything inside this directory.

An example structure for a Python file would be:

```
from isem_go2_interface import Go2
from unitree_sdk2py.core.channel import ChannelFactoryInitialize
from time import sleep

# Setup the communication with the robot
# This is the only thing that would need to be changed to switch between simulation
and the real robot
ChannelFactoryInitialize(0, "eth0")

go2 = Go2()

# We assume a starting position where the robot can raise up safely
go2.standup()
print("Stood up")

# move forward with 0.5m/s and 0.5rad/s for 2 seconds
go2.move(0.5, 0, 0.5, 2)
print("Moved!")
# move backwards with 1m/s for 2 seconds
go2.move(-1, 0, 0, 2)

sleep(5) # the robot will stand for 5 seconds with a movement command of 0,0,0

go2.damp()
print("Damp!")
# this takes 5s, to stabilize before doing anything else.
```

Let's go through this code step by step:

- The first lines import all relevant libraries, classes and functions. `isem_go2_interface` is designed by us.
- `ChannelFactoryInitialize` sets up the communication between the robot/simulator and your code. `0` and `eth0` specify the channels: the first is the domain, the second specifies the network device to broadcast messages over.
- `go2 = Go2()` creates an object resembling the Go2 robot.
- `go2.standup()` lets the robot stand up from a laying down position. See Fig 1 and 2.
- `go2.move(...)` gives the robot a velocity command that it will follow for the specified amount of seconds. This is more of a rough estimate, as the following motion is generated by a machine learning algorithm.
- `go2.damp()` is used as an emergency stop function. This resembles the command used by Unitree in the remote control.

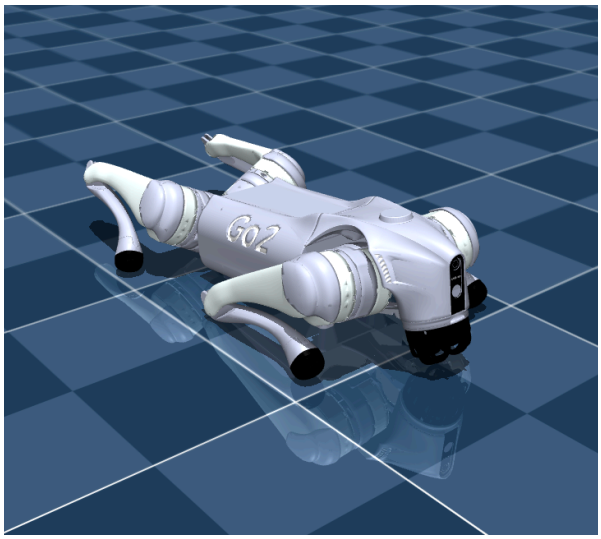

Figure 1: Laying down

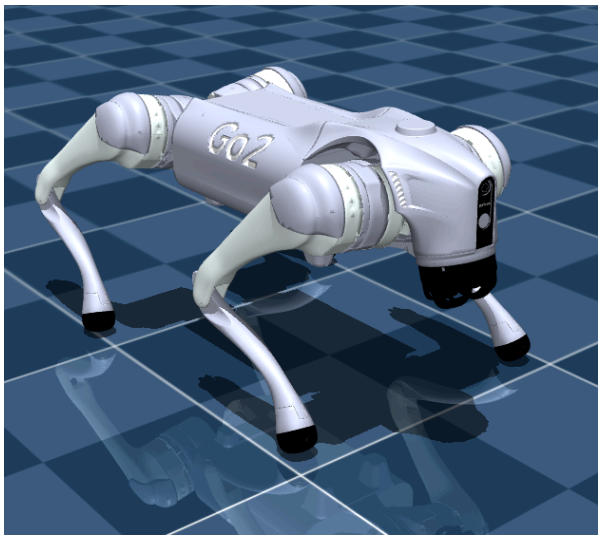

Figure 2: After standing up

> **💡 Documentation tip**
>
> Inside VSCode, hovering over functions can show their docstring. This is a built in way to document software in python projects. We implement this for our functions inside the `isem_go2_interface`. For more information into docstrings, see for example https://www.sphinx-doc.org/en/master/usage/extensions/example_google.html.

With these functions (and others you find inside the `Go2.py` file), you should be able to fully control the movement of the robot.

Some Linux basic commands

To start a Python program, you need to call it from a terminal. We collected some frequently used commands to work with terminals in Linux.

- Move to a folder: `cd [folder_name]`
  - Return to the parent folder by calling `cd ..`
- List all files and folders in the current directory: `ls`
- Install a Python package: `pip install [package-name]`
  - Be cautious with installing packages that you don't know or were given to you by an LLM. Always search for them online prior to installing, and check if their GitHub repositories or pypi.org entries look trustworthy.
- Run a Python script: `python [filename].py`
  - In some installations of Python, only `python3` instead of `python` works.
- One program typically blocks the terminal.
  - Stop the program (if it does not end by itself) with `Ctrl+C`
  - Start another program while one is running by opening a new terminal
- Copy and paste in the terminal is done by `Ctrl+Shift+C` and `Ctrl+Shift+V` respectively.